

PART A

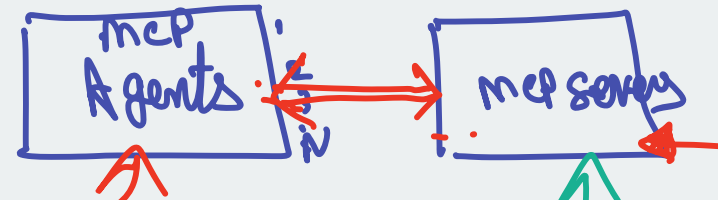
Building a Multi-Agent AI Platform

with MCP & A2A

Model Context Protocol · Agent-to-Agent Protocol · Production Architecture

The Problem: Tool Integration Chaos

Before MCP — every model needed custom glue code for every tool



BEFORE MCP

- Each agent had custom tool connectors
- Slack API code rewritten per project
- No standard for tool discovery
- **$N \text{ agents} \times M \text{ tools} = N \times M \text{ integrations}$**
- Brittle API change breaks everything
- No security model for tool access

Brittle API change breaks everything
 No security model for tool access

1. Web Search () API
 2. Estimation of Profit () API

Calculator () M=2
 Sort DB ()
 API v
 API v2



AFTER MCP

- One universal protocol for all tools
- Build once, reuse across any model
- Standardised tool discovery & schema
- **N + M integrations (not N×M)**
- Version-controlled MCP servers
- Built-in OAuth permission model

What Is a Multi-Agent AI Platform?

Definition: A system where multiple specialised AI agents, each with their own context and tools, collaborate under an orchestrator to complete complex tasks no single agent could handle alone.

Capability	Single Agent	Multi-Agent Platform
Context window	Hard limit — one model	Distributed across agents
Specialisation ✓	Generalist only	Domain experts per role
Parallelism	Sequential steps only	Concurrent sub-tasks
Tool surface	Limited by one prompt	Hundreds of MCP tools across agents
Fault tolerance	One failure = full stop	Retry at sub-agent level

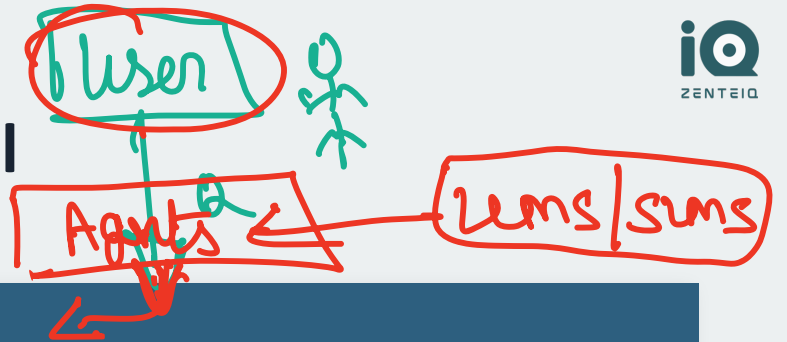
Langchain (?)

langgraph []

Why single agents hit a ceiling

MCP Architecture: Server / Client / Protocol

Model Context Protocol — Anthropic, 2024 · JSON-RPC 2.0 over HTTP-SSE or stdio



MCP CLIENT

LLM application (Claude, GPT-4o, Gemini)
Connects to servers, invokes tools, reads resources

PROTOCOL LAYER

JSON-RPC 2.0 · Capabilities negotiation · Sampling · Logging

MCP SERVER(S)

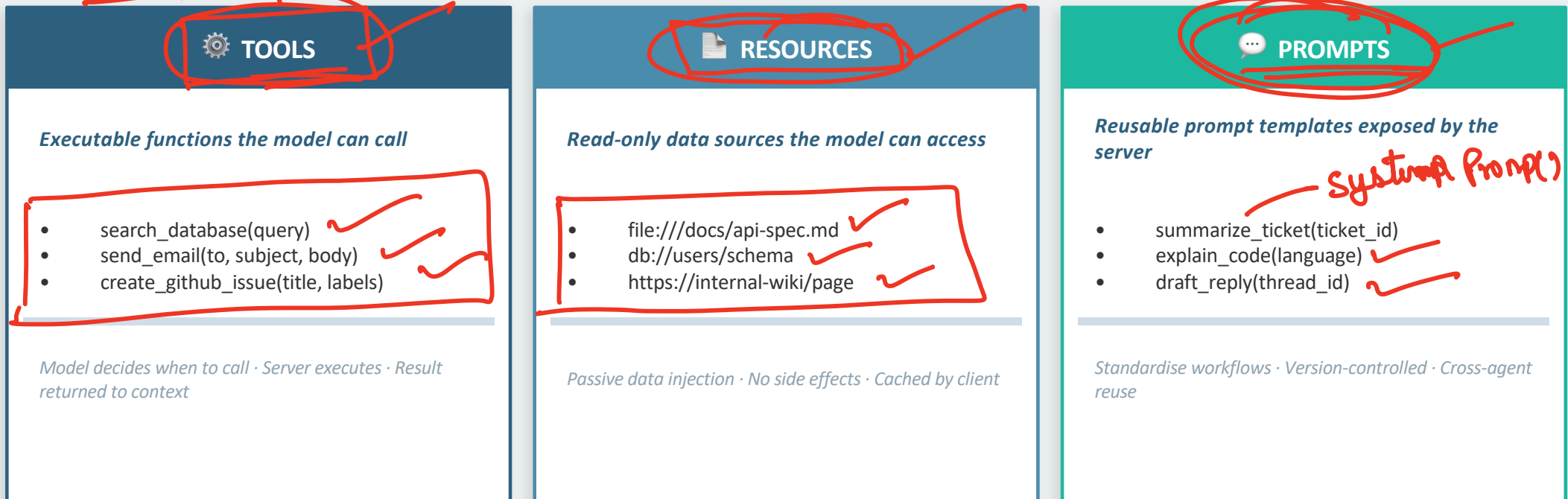
Expose: tools() · resources() · prompts()
Examples: GitHub, Slack, PostgreSQL, Google Drive, custom internal APIs

MCP Spec: modelcontextprotocol.io · Standardised since late 2024

RPC Remote Procedure call []

What an MCP Server Exposes

Three primitive types — every MCP server is composed from these building blocks



Every open-source MCP server (GitHub, Slack, Postgres...) implements these three primitives

The A2A Protocol — Agent-to-Agent Interoperability



Google Agent-to-Agent Protocol, 2024 · google.github.io/A2A

A2A: An open protocol enabling AI agents built on different frameworks (LangGraph, ADK, Agents SDK) to discover, delegate to, and receive results from each other — without knowing each other's internal implementation.

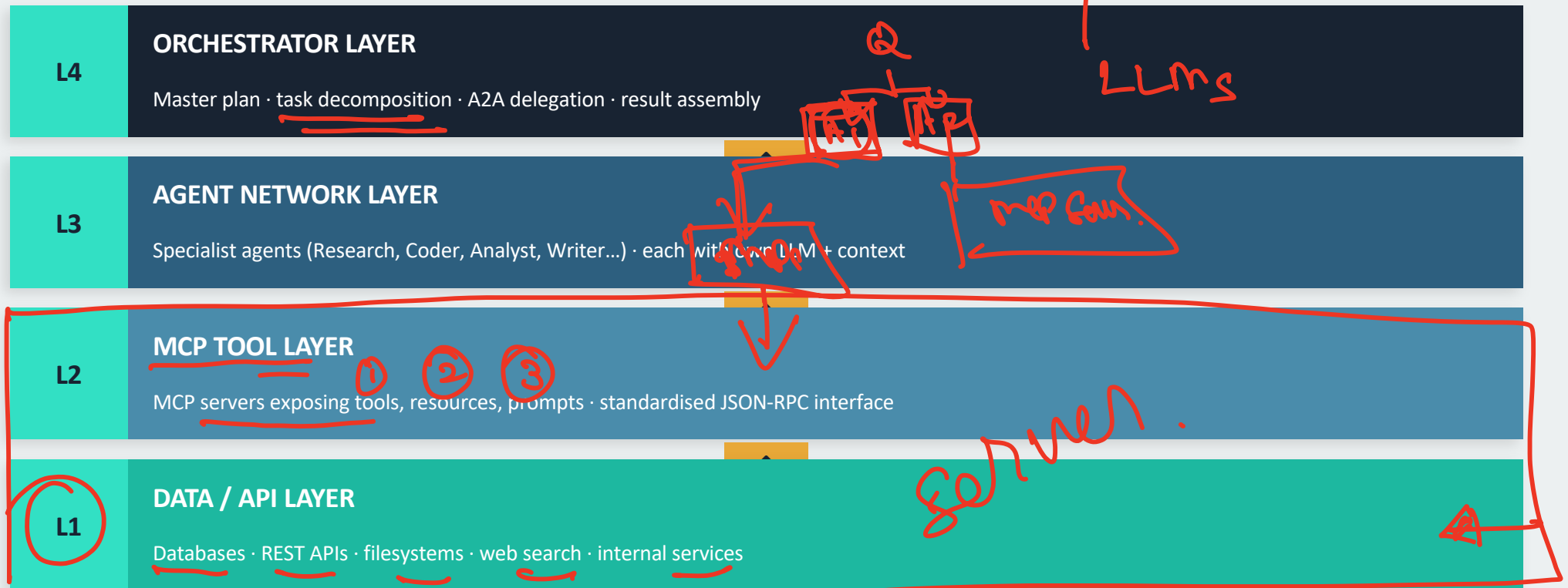
MCP vs. A2A — Two Complementary Protocols

Dimension	MCP (Model Context Protocol)	A2A (Agent-to-Agent Protocol)
Connects	Agent ↔ Tool / Data Source	Agent ↔ Agent
Direction	Tool serves the model	Peer agents collaborate
Transport	JSON-RPC 2.0 / HTTP-SSE	HTTP + structured task messages
Initiated by	LLM client calling a tool	Orchestrator delegating to worker
Key benefit	Plug-and-play tool ecosystem	Cross-framework agent interop
Analogy	USB standard for tools	API standard for agents

MCP + A2A together = the complete inter-operability stack

The 4-Layer Multi-Agent Platform Stack

Every production multi-agent platform has these four layers



A2A connects L4↔L3 · MCP connects L3↔L2 · Standard APIs at L1

Tools & Frameworks at Each Platform Layer

Layer	Primary Tool / Framework	Role	Key Feature
L4 Orchestrator	LangGraph	Stateful graph orchestration	Nodes, conditional edges, checkpointing
L3 Agent Runtime	Google ADK / OpenAI Agents SDK	Agent lifecycle + A2A handoff	Multi-turn, streaming, tool-use built-in
L3 Agent Runtime	AutoGen / CrewAI	Role-based multi-agent teams	Conversation patterns, crew roles
L2 MCP Servers	FastMCP / Anthropic SDK	Build & host MCP tool servers	Python decorator API, auto-schema gen
L2 Ecosystem	GitHub · Slack · Postgres · Drive	Pre-built MCP servers	Install & register, no custom code
Deployment	Docker + Docker Compose	Containerise each agent & server	Independent scaling, CI/CD-ready
Observability	Prometheus + Grafana	Monitor every layer	Latency, cost, tool-call rate, errors

Standard DevOps.

Tech stack as of 2025–2026 · All open-source or permissively licensed

7-Step Platform Build Walkthrough

No code — the complete conceptual sequence from blank canvas to running platform

1

Define agent roles & responsibilities

Name each agent, its domain, its inputs/outputs, and success criteria

2

Identify tools each agent needs

List every external action (search, write, query...) → map to MCP server candidates

3

Register or build MCP servers

Use existing ecosystem servers; build FastMCP servers for proprietary APIs

4

Wire agents to their MCP servers

Each agent client points to its authorised set of MCP servers (least-privilege)

5

Build the LangGraph orchestrator

Define the state graph: nodes = agents, edges = delegation conditions (A2A messages)

6

Add security & guardrails

OAuth tokens, output sanitisation, permission gates, human-in-the-loop checkpoints

7

Containerise & deploy

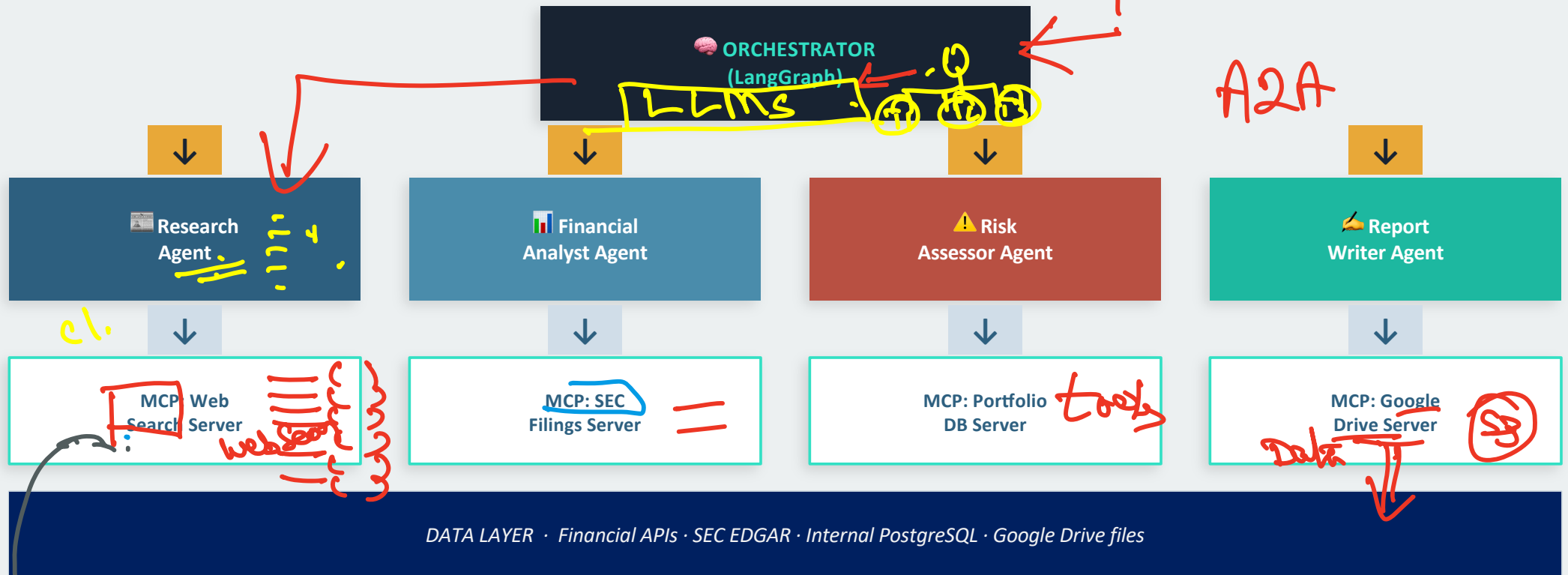
One Docker container per agent + per MCP server; Docker Compose for local; K8s for prod

Steps 1–3: Design phase · Steps 4–5: Integration phase · Steps 6–7: Production phase

EXAMPLE

Example Platform: Investment Research System

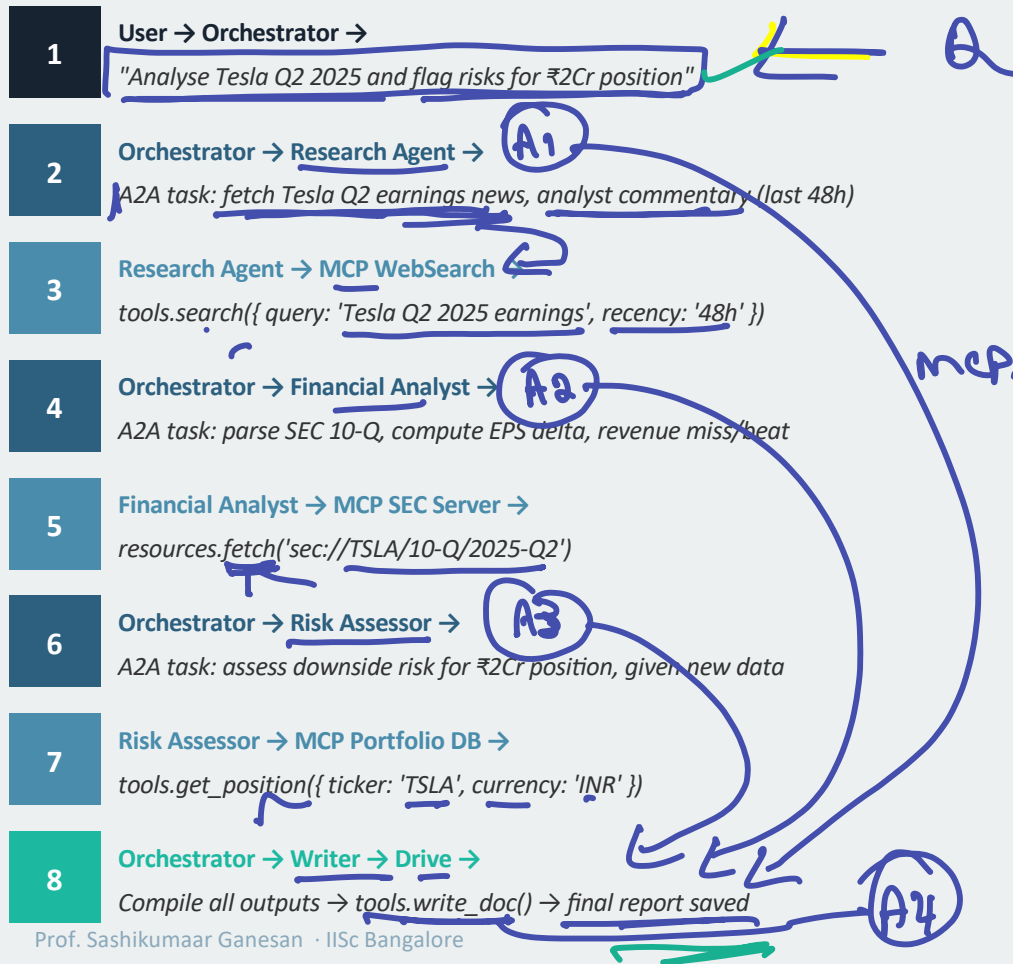
User query: "Analyse Tesla Q2 2025 earnings and flag risks for our ₹2Cr position"



DATA LAYER · Financial APIs · SEC EDGAR · Internal PostgreSQL · Google Drive files

4 specialist agents + 4 MCP servers + 1 orchestrator = complete research platform

Message Flow: From User Query to Final Report



Build Steps 1–3: Design → Tool Mapping → MCP Registration

Applied to the Investment Research Platform example

STEP 1 DEFINE AGENT ROLES

- Research Agent: domain = news & filings, output = structured JSON summaries
- Financial Analyst: domain = earnings & ratios, output = EPS / revenue delta
- Risk Assessor: domain = portfolio risk, output = VaR + risk flags
- Report Writer: domain = synthesis, output = PDF-ready markdown

STEP 2 MAP TOOLS TO AGENTS

- Research → web_search, article_fetch → WebSearch MCP server
- Analyst → sec_fetch, earnings_parse → SEC Filings MCP server
- Risk → get_position, compute_var → Portfolio DB MCP server
- Writer → write_doc, save_file → Google Drive MCP server

STEP 3 REGISTER MCP SERVERS

- WebSearch: use Brave Search MCP (open-source, 1-line install)
- SEC Server: build custom FastMCP server; decorate SEC EDGAR REST calls
- Portfolio DB: build FastMCP wrapper around internal PostgreSQL
- Google Drive: use Anthropic reference MCP server (pre-built)

Agents ← Servers ← Tools

Dev

OPS

Build Steps 4–7: Wire → Orchestrate → Secure → Deploy

From integration through to containerised production deployment

STEP 4 WIRE AGENTS → MCP SERVERS

- Each agent's MCP client config lists its authorised servers
- Research Agent: allowed = [websearch-mcp]; blocked = [portfolio-mcp]
- Least-privilege: agents see only the tools they need

STEP 5 BUILD LANGGRAPH ORCHESTRATOR

- State = { task, results{}, flags{} }
- Nodes: research_node, analyst_node, risk_node, writer_node
- A2A edges: conditional on previous node output status

STEP 6 SECURITY & GUARDRAILS

- OAuth 2.0 bearer tokens per MCP server
- Output sanitiser: strip injected instructions from tool results
- Permission gate: human approval before any trade action

STEP 7 CONTAINERISE & DEPLOY

- 1 Dockerfile per agent + 1 per MCP server
- Docker Compose: all services + Redis + Prometheus
- Health checks on every container; CI/CD on push

Steps 4–7 = Integration + Production phase — agent wiring, graph design, security, containers

Security Model: OAuth + Least Privilege + Injection Defense

Three attack surfaces — three corresponding defences

TRANSPORT & AUTH

Attack surface:

- Eavesdropping on agent↔tool comms
- Rogue MCP server impersonation
- API key leakage in logs

Defence:

- TLS/HTTPS for all remote MCP servers
- OAuth 2.0 bearer tokens per server
- Process isolation for local stdio servers

TOOL PERMISSIONS

Attack surface:

- Agent calls tools beyond its role scope
- Excessive data read (GDPR risk)
- Agent writes/deletes without approval

Defence:

- Capability declarations in server manifest
- Least-privilege: allowlist per agent
- Human-in-the-loop gate for destructive actions

PROMPT INJECTION

Attack surface:

- Malicious tool result hijacks agent
- "Ignore previous instructions" in web content
- Chained injection across agent calls

Defence:

- Output sanitiser strips instruction patterns
- Sandboxed tool execution environment
- Restrict dynamic tool discovery scope

Ref: AI-OP-IN-TH-000011 · MCP Security, Permissions, and Composability

MCP + A2A vs. Single Agent: When to Use What

Signal	Use Single Agent + MCP	Use Multi-Agent Platform (MCP + A2A)
Task complexity	One clear goal, ≤ 5 tool calls	Multi-step, cross-domain, 10+ tool calls
Context window	Fits in a single model's context	Requires distributed context across roles
Parallelism needed?	Sequential is fine	Sub-tasks can run concurrently $\rightarrow$ speed gain
Domain expertise	Generalist LLM is sufficient	Each sub-task needs specialist tuning/prompting
Fault tolerance	Low-stakes, failure = retry	Mission-critical, partial failure must not cascade
Team size	Solo developer / prototype	Engineering team, production system
Time to build	Hours to days	Days to weeks (worth it for complex systems)

Rule of thumb: start with a single agent; migrate to multi-agent when you hit context or parallelism limits

Building a Multi-Agent AI Platform

- ① MCP solves the N×M integration problem — one protocol, plug-and-play tool ecosystem
- ② A2A enables cross-framework agent interoperability — orchestrate agents built on different SDKs
- ③ The 4-layer stack: Orchestrator (LangGraph) → Agents → MCP Servers → Data APIs
- ④ 7 concrete steps: Design roles → Map tools → Register MCP → Wire agents → Graph → Secure → Deploy
- ⑤ Investment Research example: 4 agents + 4 MCP servers coordinate via A2A delegation and tool calls
- ⑥ Security is a first-class concern: TLS, OAuth 2.0, least-privilege, prompt-injection sanitisation

Part B — Module 9: LLMOps & Production Agentic Systems → follows